

Twinkle — database design

SQLite, one file, sixteen tables. Written from the model as it stands, not from intent — if this document and `src/Pharma.Core/Entities.cs` disagree, the code is right and this is stale.

Upgrading a live database is a separate subject: [DATABASE_UPGRADES.md](#).

Where it lives

Database	<code>C:\HMS\DB\twinkle.db</code>
Backups	<code>C:\HMS\DBBackup\</code> — daily, plus one before every schema change
Logs	<code>C:\HMS\Logs\</code>

One file, outside the program folder on purpose: an upgrade replaces `C:\HMS\App` and must not be able to touch the data. A clinic can copy `twinkle.db` to a pen drive and that is the whole backup.

`C:\HMS` rather than `%ProgramData%` because a clinic can find it, and the person helping them over the phone can say the path out loud.

Conventions

Every table inherits `BaseEntity` :

Column	Type	Notes
<code>Id</code>	TEXT	GUID, generated in the application, not by the database
<code>CreatedAt</code>	TEXT	Stamped in <code>SaveChanges</code> , never by the caller
<code>UpdatedAt</code>	TEXT null	Stamped on modify
<code>IsDeleted</code>	INTEGER	Soft delete

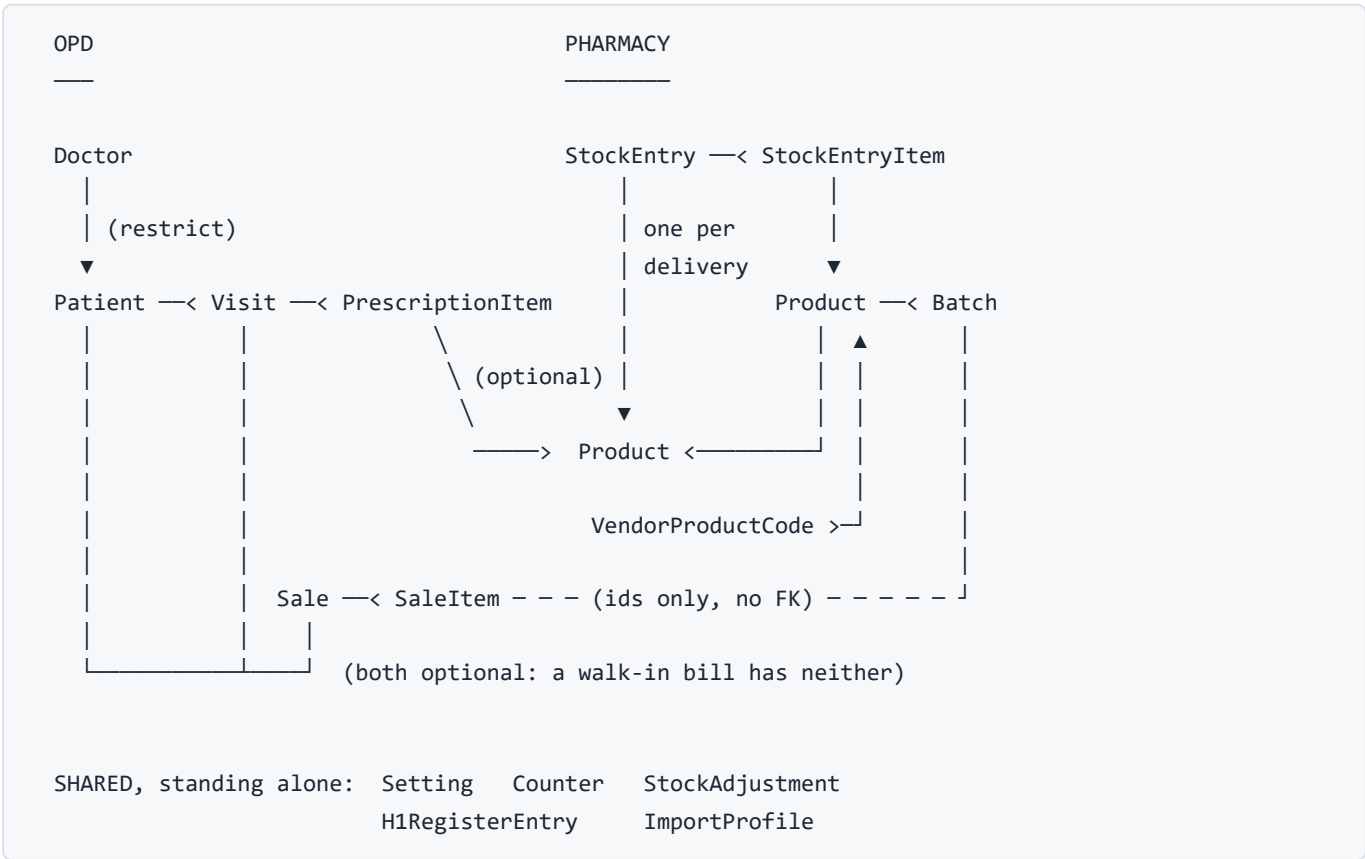
GUID keys, not integers. Nothing needs a human-readable row number — the documents carry their own numbers (`PatientNo` , `BillNo` , `VisitNo`) from the `Counters` table. GUIDs also mean two databases could be merged one day without a key collision, which an autoincrement forecloses.

Soft delete, filtered explicitly. There is deliberately **no global query filter**. Services write `.Where(x => !x.IsDeleted)` themselves, because EF warns on required navigations into filtered entities and the explicit clause is easier to follow in a codebase this size. The cost is that a forgotten `where` shows deleted rows; the benefit is that nothing is hidden by magic.

Money is `decimal(12,2)` everywhere, applied in one loop in `OnModelCreating` rather than attribute by attribute. Twelve digits is far more than a clinic needs and keeps SQLite storage predictable.

Local time, not UTC. A single-PC clinic in one timezone; the day book is "the day the shop had", and UTC would put late-evening sales on tomorrow's report. This would need revisiting the moment there is a second branch.

The shape of it



`SaleItem` holds `ProductId` and `BatchId` as plain columns with **no foreign key** — see [Bills do not point at anything](#).

OPD

Patient

`PatientNo` (unique), `Name`, `Phone`, `Gender`, `Age`, `Address?`, `Allergies?`

Indexed on `PatientNo` (unique), `Phone`, `Name` — the desk searches by name or phone and `Phone` is deliberately **not** unique, because a family shares one number and all the children have to come back from a search.

`Age` is stored as a number, not a date of birth. Wrong for a paediatric clinic in the long run — a two-year-old's record is stale in a year — but it is what the desk is given at the counter, and changing it is a migration plus a UI change, recorded here rather than quietly fixed.

Doctor

`Name` , `RegistrationNo?` , `Speciality?` , `Phone?` , `ConsultationFee` , `IsActive`

`ConsultationFee` is the default that fills in when booking; the fee actually charged lives on the `Visit` , so changing a doctor's rate never rewrites history.

Visit

A booking and a visit are the same row. Turning up moves `Status` `Booked` → `Waiting` , so the desk never re-keys anything.

`VisitNo` (unique), `TokenNo` , `ScheduledOn` (indexed), `Status` , vitals (`WeightKg?` , `BloodPressure?` , `TemperatureF?`), `Complaint?` , `Diagnosis?` , `Notes?` , `FollowUpOn?`

The fee is a small ledger of its own: `Fee` , `FeePaid` , `FeeReceiptNo?` , `FeePaidOn?` , `FeePaymentMode?` . The receipt number and date are stored rather than derived so a receipt can be reprinted years later and reconciled against the day's collection.

PrescriptionItem

`MedicineName` , `Dosage?` , `Frequency?` , `Days` , `Quantity` , `Instructions?`

`ProductId` is **nullable** on purpose. A doctor prescribes what the child needs, whether or not this pharmacy stocks it — a free-text line is printed on the prescription and never becomes a medicine in the catalogue. That nullable column is the whole feature.

Pharmacy

Product — what a medicine *is*

Set up once. Holds no stock.

`Name` , `GenericName?` , `Manufacturer?` , `Composition?` , `Storage?` , `PackSize?` , `DispensingUnit` , `UnitsPerPack` , `AllowLooseSale` , `HsnCode` , `GstRate` , `Schedule` , `RackLocation?` , `ReorderLevel` , `IsActive` , `SearchKey`

`UnitsPerPack` **is the most consequential column in the database.** Stock is counted in sellable units — tablets, not strips — so part of a strip can be sold. Set it to 1 while the pack size says "15 TAB" and the counter charges fifteen times the price for every tablet, and nothing anywhere reports an error. It is clamped to a minimum of 1 in the property getter, because rows written before that guard existed hold 0.

SearchKey is brand + maker + pack, normalised — trimmed, case-folded, whitespace collapsed — with a **unique index filtered on IsDeleted = 0**. It stops the same medicine existing twice, which otherwise splits its stock across two rows and shows up twice at the counter. Stored as a column rather than checked in the screen because a database constraint is a guarantee and a screen check is a suggestion. The filter is what lets a deleted medicine's name be used again.

Storage exists because this is a children's clinic: vaccines, insulin and some syrups are ruined by a warm shelf.

Batch — what is actually on the shelf

ProductId, BatchNo, ExpiryDate, Mrp, PurchaseRate, QtyOnHand, UnitsPerPack, SupplierName?, SupplierInvoiceNo?, FreePacks, PacksReceived, ReceivedOn, IsProvisional

Indexed on (ProductId, BatchNo).

Stock is always batch-wise. MRP and expiry are printed on the strip and differ between consignments of the same drug, so both have to come from the batch and never from the product. A bill line therefore prices against the batch it came out of, and a course spanning two batches is two lines at two prices.

UnitsPerPack is duplicated here on purpose. It is snapshotted from the product at receiving, because a manufacturer changes pack size between consignments and old stock must keep pricing against the pack it actually came in.

FreePacks is the "+1" in 10+1. It was captured at receiving and then thrown away, which overstated cost per unit on every margin figure — ten paid for and eleven received makes the real cost $\text{rate} \times 10 \div 11$.

IsProvisional marks stock keyed at the counter because the medicine was on the shelf but not in the system. Purchases will not tie out against sales until it is matched to a supplier bill, so it is **flagged rather than hidden** — that is the *Stock to reconcile* report.

StockEntry and StockEntryItem — the goods-inward document

EntryNo (unique), EntryDate, SupplierName?, SupplierInvoiceNo? (indexed), TotalAmount, NetAmount, DiscountPercent, Notes?, ImportedFile?, ImportProfile?, IsProvisional, EnteredBy?

Items carry **Quantity in packs, as the vendor billed them**, with **UnitsPerPack** multiplying into base-unit stock. This is the one place where packs and units meet, which is why the receiving screen reads the conversion back out loud: "20 pack(s) × 15 = 300 tablets onto the shelf".

SupplierInvoiceNo is indexed because it is what stops the same bill being imported twice.

Sale and SaleItem — the bill

BillNo (unique), BillDate (indexed), CustomerName, DoctorName?, GrossAmount, DiscountAmount, TaxableAmount, CgstAmount, SgstAmount, RoundOff, NetAmount, PaymentMode, Status, IsTaxInvoice

PatientId and **VisitId** are both **nullable** — most counter sales are walk-ins with no patient record at all. **PatientId** deletes with **SetNull**, so removing a patient cannot take their bills with them.

`IsTaxInvoice` is stored on the bill, not read from settings. A reprint years later has to show what was actually given to the customer, even if the clinic has registered for GST since. Note it is not the same as "carries tax": a registered clinic selling only zero-rated goods still issues a tax invoice.

Bills do not point at anything

`SaleItem` keeps `ProductId` and `BatchId` as plain columns with **no foreign key**, and copies `ProductName`, `BatchNo`, `ExpiryDate`, `HsnCode`, `UnitsPerPack`, `Mrp`, `GstRate` and `PackLabel` onto itself.

This is deliberate denormalisation, and it is the right call: **a bill is a document, not a view**. What was handed to the customer must reprint identically in three years, whatever has happened to the medicine record since — renamed, repriced, repacked, deleted. A foreign key would also stop a batch ever being tidied away.

The cost is that a medicine renamed today does not change on yesterday's bill. That is not a bug; that is the feature.

Shared

Counter — document numbering

`Name` (unique), `Prefix`, `LastNumber`

`INV00001`, `P00012`, `V00043`. Tax invoice numbers must run without gaps, so they come from a row that is read and incremented in the same transaction as the document, rather than from a count of existing rows — which would reuse a number after a deletion.

StockAdjustment — the correction trail

`AdjustedOn` (indexed), `BatchId`, `ProductId`, `ProductName`, `BatchNo`, `QuantityBefore`, `QuantityAfter`, `Reason`, `Notes?`, `AdjustedBy?`

Stock otherwise only moves by receiving or selling, and both leave a document. A manual correction has none, **so it writes one** — otherwise a shortfall is indistinguishable from theft and nobody can answer what happened.

`ProductName` and `BatchNo` are copied for the same reason as on a bill: the trail has to survive later edits.

H1RegisterEntry — statutory

`SoldOn`, `BillNo`, `ProductName`, `BatchNo`, `Quantity`, `PatientName`, `DoctorName?`

Schedule H1 sales, retained three years by law. A separate table rather than a query over sales, because it is a register that must be producible on demand and must not change when a bill is edited or a medicine reclassified.

Setting — shop identity

Key (unique), Value

Key-value, in the database rather than `appsettings.json`, so a second branch is a data change rather than a redeploy. It is also why a ClickOnce update replacing `appsettings.json` costs nothing — none of the clinic's identity is in there.

ImportProfile and VendorProductCode

`ImportProfile` holds one supplier's file format: column names, date formats, how their quantities are expressed. `VendorProductCode` maps a vendor's own code to our product, uniquely per (`VendorProfile`, `Code`) — two suppliers call the same drug `000071` and `31435`, so the mapping cannot be global. Recording it on the first import is what makes every later import of that vendor match unasked.

Delete behaviour

Relationship	On delete	Why
Visit → Patient	Restrict	A patient with visits cannot be removed; the history is the point
Visit → Doctor	Restrict	Same — past visits name the doctor who saw them
Visit → PrescriptionItem	Cascade	A prescription has no meaning without its visit
Batch → Product	Restrict	Never lose stock by tidying the catalogue
StockEntry → items	Cascade	Lines belong to the document
Sale → items	Cascade	Same
Sale → Patient	SetNull	Removing a patient must not delete their bills
VendorProductCode → Product	Cascade	A mapping to a deleted product is noise
StockAdjustment → batch, product	Restrict	The audit trail outlives tidying up

Indexes

Table	Index	For
Patients	PatientNo unique	The number on the card
Patients	Phone , Name	The two ways the desk searches
Visits	VisitNo unique	

Table	Index	For
Visits	ScheduledOn	The day's queue, every screen refresh
Products	Name	Counter search, as-you-type
Products	SearchKey unique, filtered IsDeleted = 0	No duplicate medicines
Batches	(ProductId, BatchNo)	FEFO allocation and receiving
Sales	BillNo unique, BillDate	Reprint, and the day book
StockEntries	EntryNo unique, SupplierInvoiceNo	Stops a bill importing twice
StockAdjustments	AdjustedOn	The corrections list
Settings, Counters, ImportProfiles	Key / Name unique	
VendorProductCodes	(VendorProfile, Code) unique	Per-vendor mapping

Things that look like columns and are not

Read the database directly and these will be missing — they are computed in the application and explicitly ignore d:

- **Product** — StockOnHand (sums its batches), PackDescription, UnitPriceLabel, PackPriceLabel, RackLabel, Shortage, Level
- **Batch** — IsExpired, UnitPrice, OnHand, Returnable, DaysToExpiry, EffectivePackCost, Display
- **Visit** — IsWaiting, PatientLine, FeeBadge, RowSummary, WaitedFor
- **SaleItem** — QuantityDescription
- **StockEntryItem** — LineTotal, UnitsReceived
- **StockAdjustment** — Change, Direction

StockOnHand is the one to know about: **there is no stock column on Product**. Stock is the sum of its batches, always, so it cannot drift out of step with them.

What this schema does not do

Recorded so nobody has to discover it:

Not supported	Note
Sales returns	No credit note, no reverse entry. Parked by decision; do it on paper and correct the count with a reason

Not supported	Note
A stock ledger	<code>QtyOnHand</code> is a running figure. Adjustments are recorded, but there is no single movement table to reconstruct the shelf on an arbitrary past date
IGST / inter-state	CGST + SGST only. A retail counter almost never raises an inter-state invoice
A supplier master	Supplier is free text on the batch and the entry. Means the same supplier can be spelled three ways
Users, logins, permissions	Everyone using the PC has the same rights. <code>EnteredBy</code> and <code>AdjustedBy</code> hold the Windows username, which is a note, not a control
More than one PC	One file, one till. No concurrency design beyond SQLite's own locking
Date of birth	<code>Age</code> is a number, entered once. Stale within a year for an infant
Multiple schedules per medicine	<code>DrugSchedule</code> is one value. A real bill shows <code>G & H</code> — see BILL_REVIEW.md

Changing it

Never by hand, never with a SQL script run at a clinic. Add a migration:

```
dotnet ef migrations add DescribeTheChange --project src\Pharma.Data --startup-project src\Pharma.Data
```

The rules, the tests behind the upgrade path, and what SQLite makes awkward are in [DATABASE_UPGRADES.md](#).